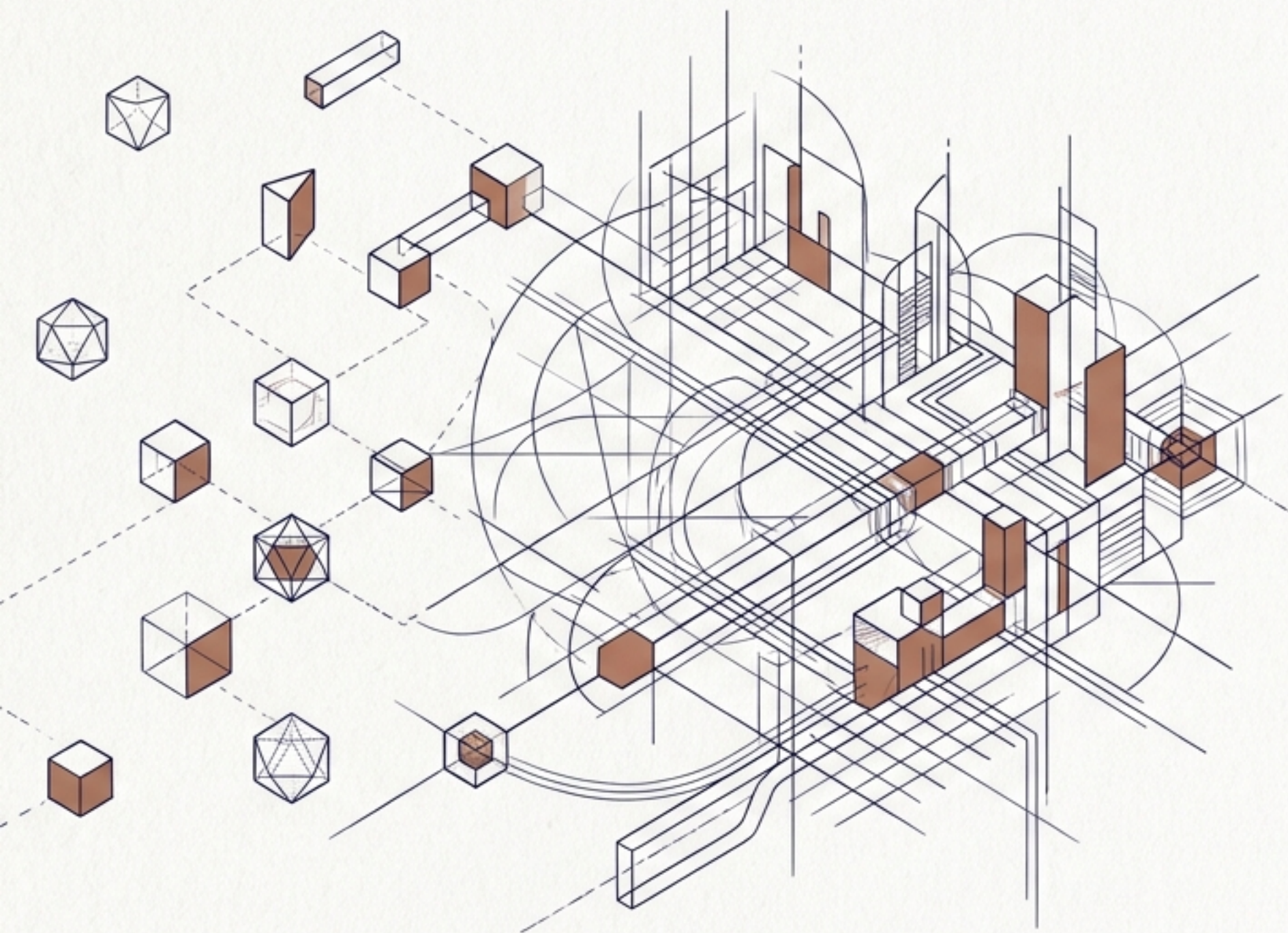
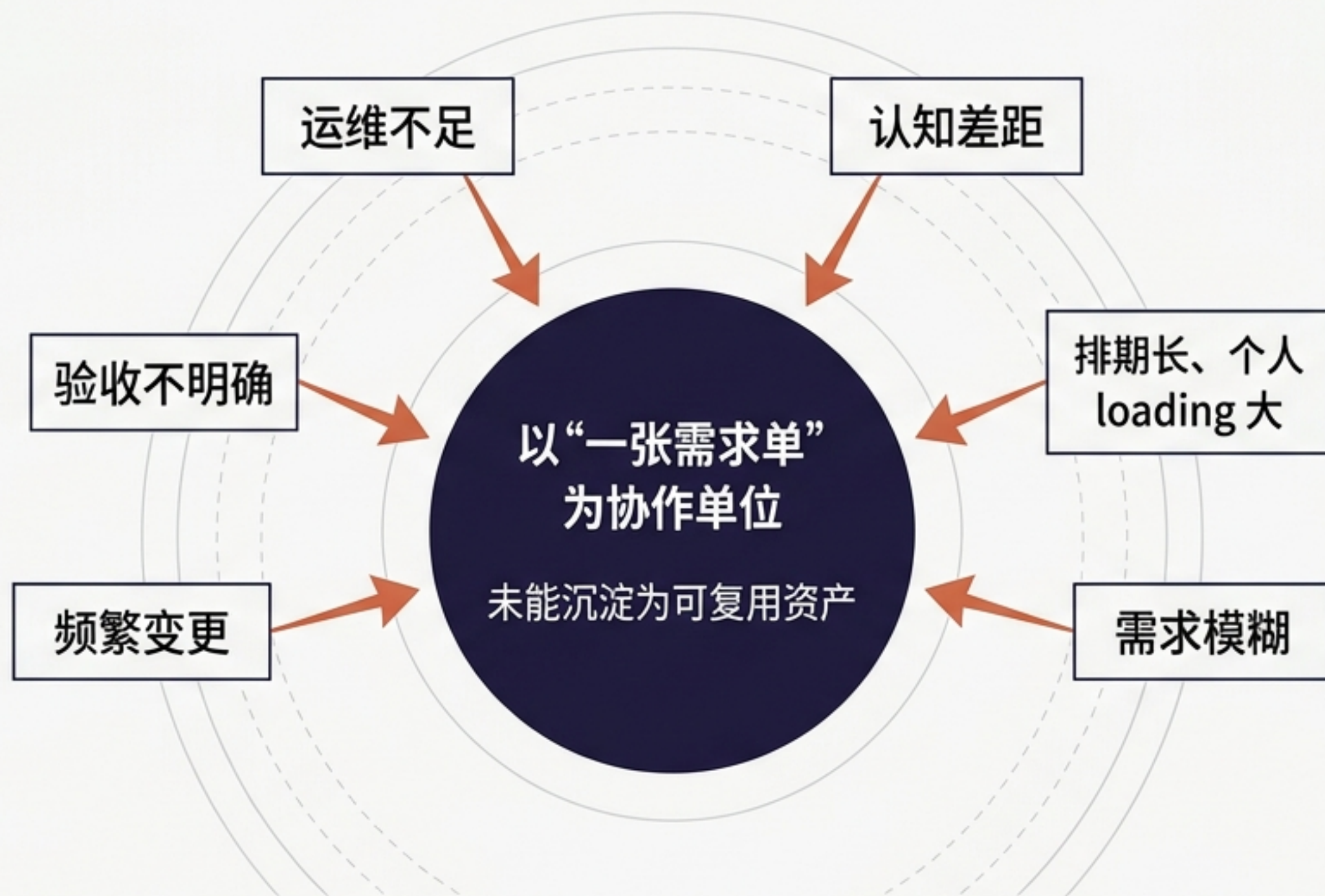




从“接需求”到 “共同经营能力”

算法开发与业务协作的六个演进命题

[Confidential / Internal Discussion Blueprint]



四次翻译损耗：每次新需求都在重新解释、寻找和定义。AI的核心价值是减少这些损耗

业务现象

可定义的问题

可复用的技术能力

可验证的业务结果

人类与 Owner (价值与边界)

- 决定业务价值与优先级
- 承诺技术可行性、资源与排期

AI Agent (编排与翻译)



Rule: AI绝不能替业务/技术Owner做决策

AI Agent (编排与翻译)

- 补充信息与调查事实
- 暴露矛盾与编排变化

Rule: Agent是连接器，不是掩盖底层混乱的创可贴

底层系统 (唯一事实源)

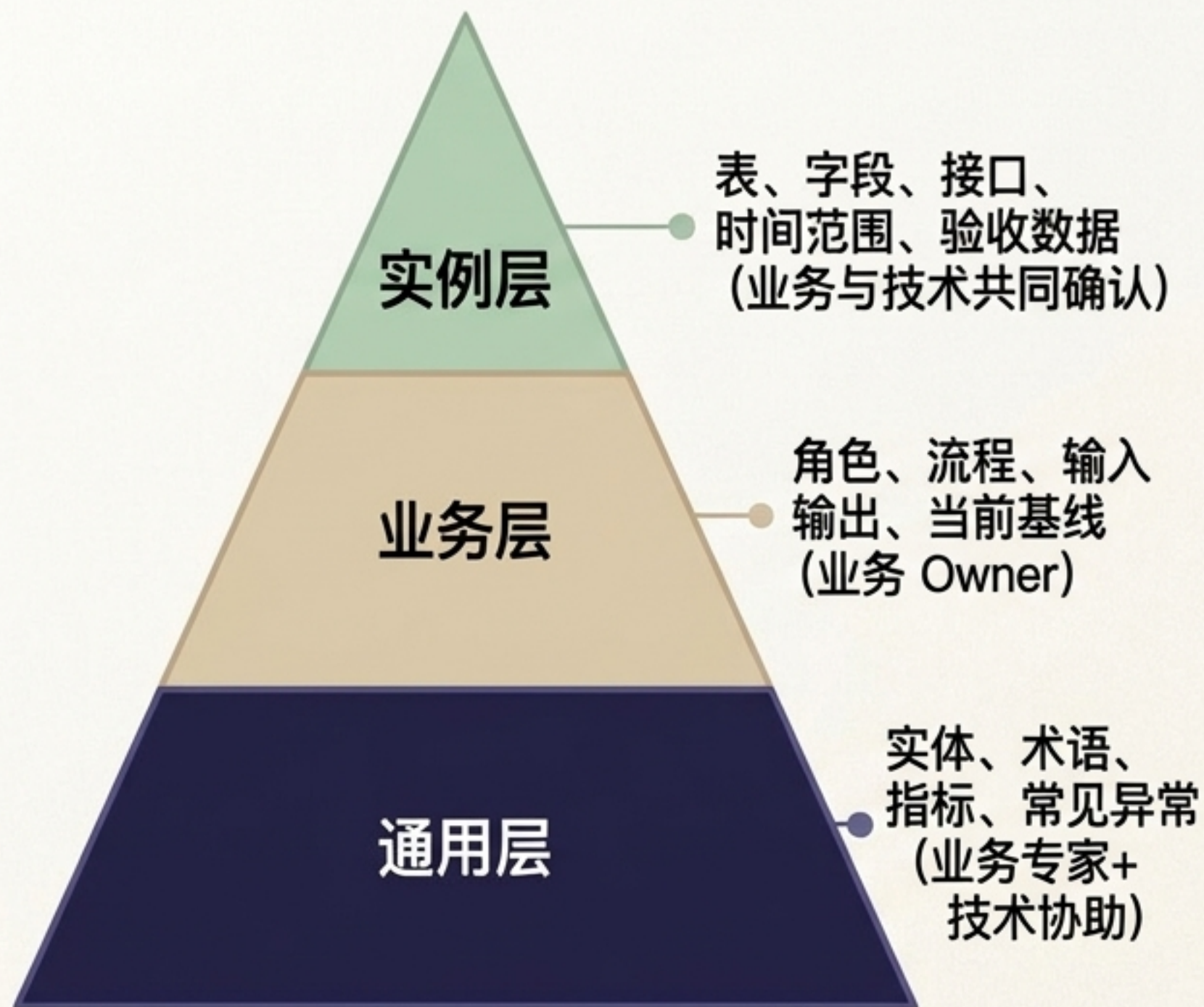
- API、代码、规则、权限与数据定义
- 知识与需求状态的载体

Rule: Agent不应成为新的
事实源

协作闭环的六个断点映射

断点（问题）		演进方向（解决方案）
认知差距：共同语义缺失	➡	SOP → 业务技术映射 → 实例上下文包
排期与高负载：缺乏自助分诊	➡	人工咨询 → 有依据的问答 → 自助处理与升级
需求模糊：缺乏决策状态	➡	口头描述 → 结构化澄清 → Ready Gate
频繁变更：缺乏稳定/变化边界	➡	一需求一开发 → API/Tool 能力 → Agent 组合
验收不明确：缺乏证据契约	➡	“效果不错” → 基线、指标、验证与责任人
运维不足：缺乏自助能力	➡	开发人工兜底 → Runbook/API → 运维 Agent

这不是六套独立的系统，而是一套协作闭环的演进方向。

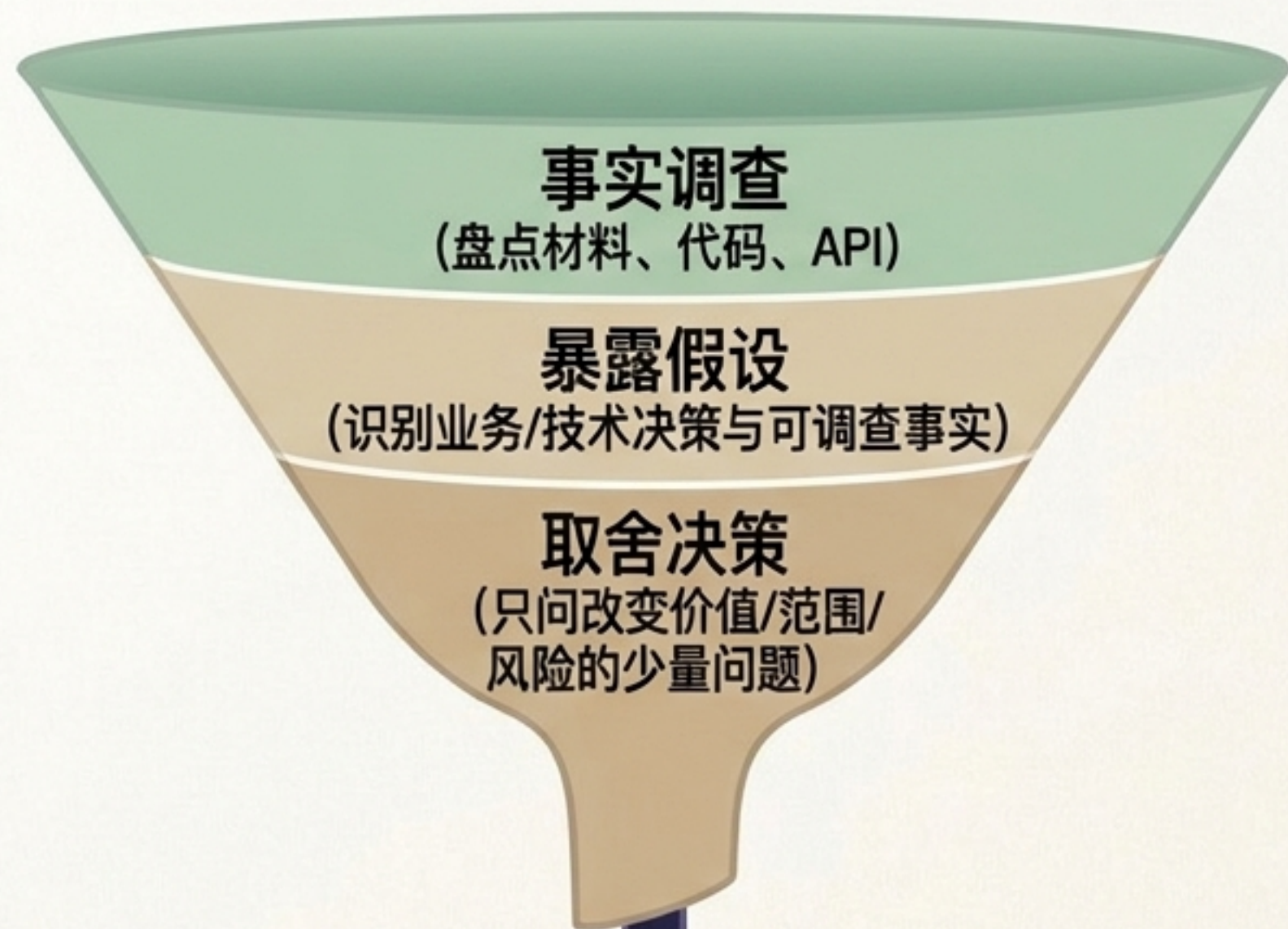


缺的不是文档，
而是机器与人都能懂的共同语义包



分诊机制拦截高频咨询，释放专家时间，
而非做一个什么都敢答的过期机器人

Agent 的评估与追问



**Rule: AI 不能替缺席方
思考或裁决价值取舍**

ONES 承载的 Ready 状态

confirmed
(已确认)

assumption
(临时假设)

needs-decision
(待业务取舍)

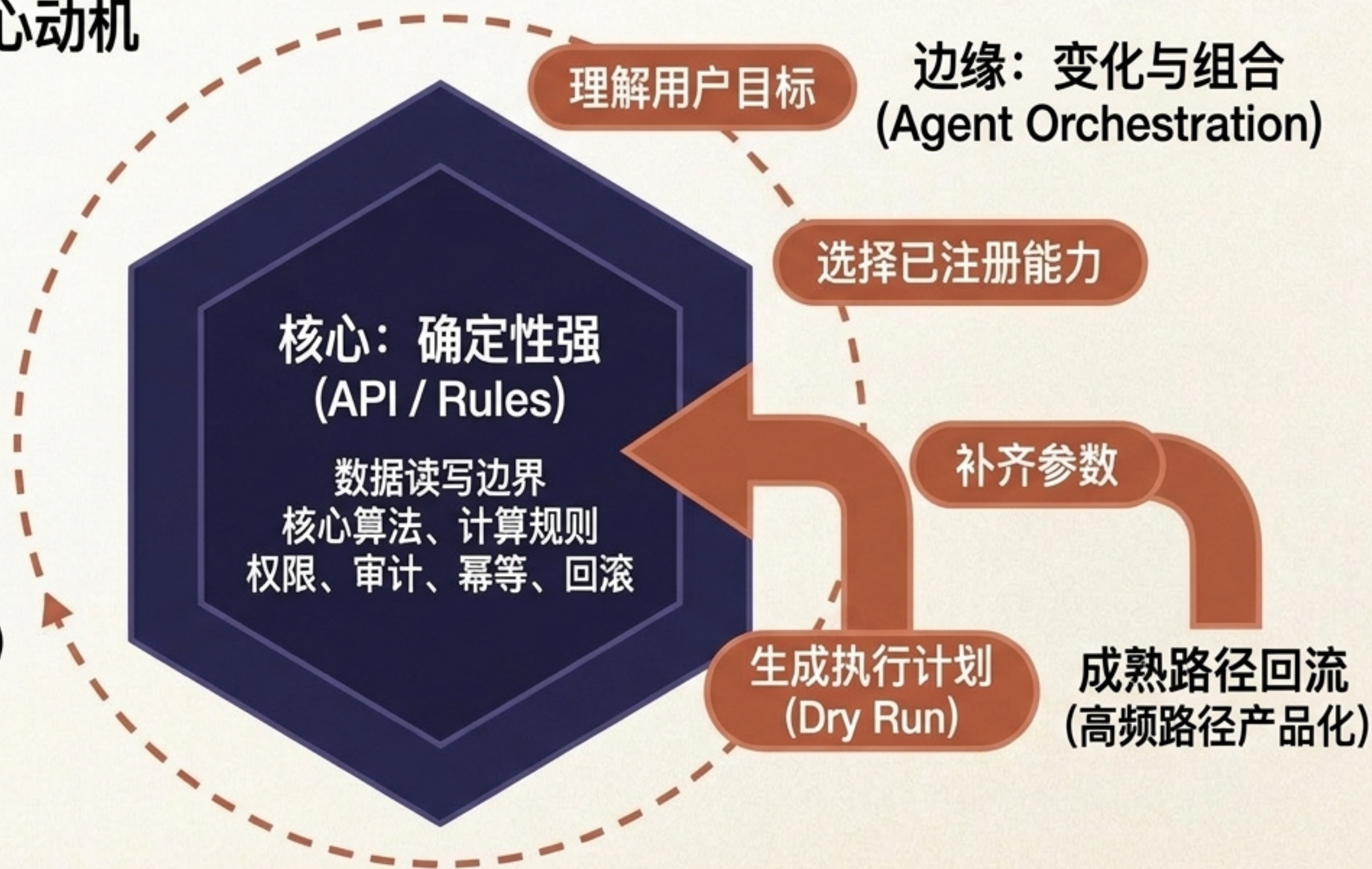
needs-evidence
(待技术证据)

needs-spike
(待最小试验)

Agent负责语义调查，ONES系统负责承载Ready状态，输出明确结论而非掩盖阻塞项的总分

核心：确定机心动机

边缘：变化与组合
Agent
(Agent Orchestration)

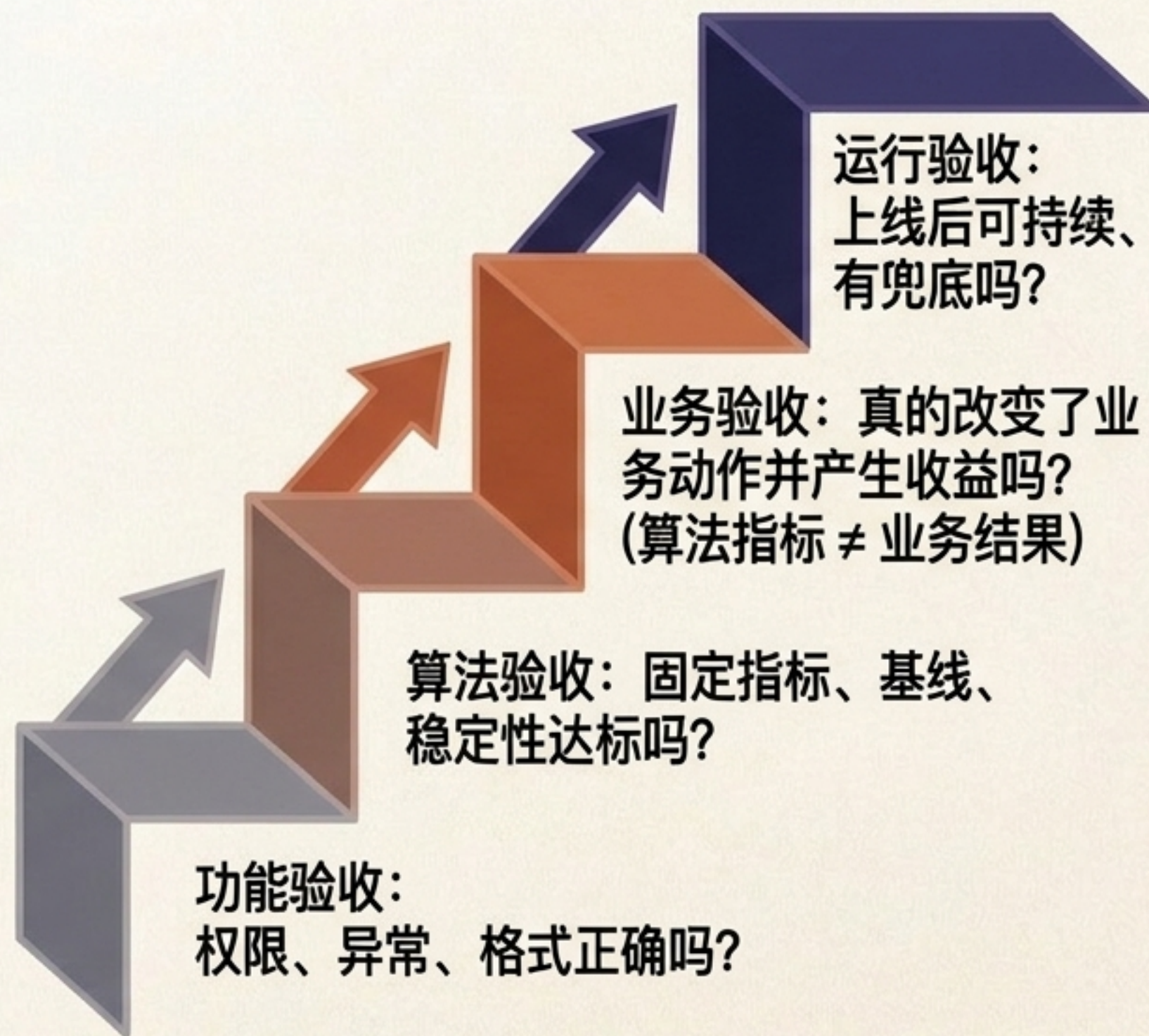


必须有回流机制，防止 Agent 沦为不可测试的自然语言胶水

双重收益评估矩阵



四层验收阶梯



没有权限管控和回滚机制，Agent 化只是把人工风险换成了自动化灾难



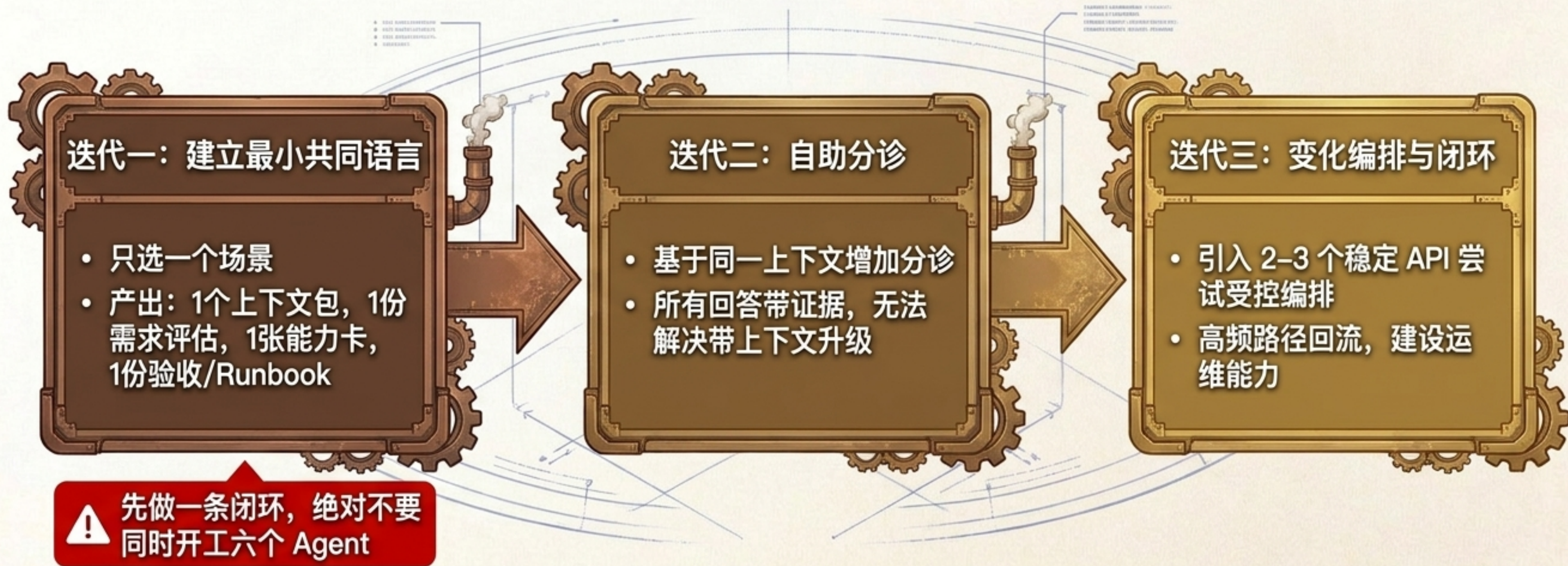
五大共享权威资产与六大中心化Agent交互图



核心原则

绝不能建 6 个维护各自状态的独立 Agent。
Skill 只保留流程知识，事实必须从中央资产库读取。

务实演进：三步走战略



⚠ 先做一条闭环，绝对不要同时开工六个Agent

我们量度什么？(Success Metrics that Matter)

真实解决率

不是“问答拦截率”（用户可能只是放弃），而是有证据的一次解决率。

升级无损度

转人工时，上下文完整率有多高？业务和技术是否不再需要“从头再说一遍”？

核心资产复用率

一个沉淀下来的能力契约/API，实际被多少个真实业务场景复用？

零痛点运行

上线后的人工兜底次数、故障恢复时间，以及真实的业务采用率。

必须直面的非共识判断

更强的模型解决不了认知差距。如果没有可确认的共同语义，大模型只是在更流畅地猜测。

频繁变更不应全部 Agent 化。Agent 只适合承接长尾变化；一旦高频稳定，必须重新固化为产品能力。

问答 Agent 可能增加负担。没有 Owner 和升级机制的机器，只会变成一套需要开发人员维护的新系统。

算法指标 $\neq$ 业务收益。真正的业务业务收益不是准确率提升，而是决策或流程发生了可验证的改善。

第一步怎么走？

1. 六个问题中，哪个目前消耗的有效开发时间最多？（需数据支撑）
.....
2. 哪个真实场景同时满足“高频、有 Owner、有接口基础”，最适合做最小闭环？
.....
3. 当前哪些知识其实已经存在，只是缺少业务到技术的映射？
.....
4. 哪三类决策我们明确只能由业务/技术 Owner 做，绝不交给 AI？
.....
5. 第一阶段，我们承诺“绝对不做什么”，以避免变成盲目的平台造轮子？
.....